

API 서버 보안 · RBAC

Kubernetes in Action · 1판 12장

핵심: HTTPS · API 토큰 · RBAC · ABAC · ReBAC

정리: `CHAPTER12-Study.md` · 실습: `ch12-minikube/`

1판 vs 2판 — 장 번호 주의

주제	1판 12장	2판 12장
API 인증·RBAC·ServiceAccount	오늘	—
Ingress	이후 장	12장
Downward API	8장	9장

2판 목차 — 같은 번호, 다른 내용

오늘 다룰 내용 (12장 전체)

절	주제
12.1	Users · ServiceAccount · 생성 · Pod 할당 · imagePullSecrets
12.2	RBAC — Role/Binding · cluster 리소스 · 기본 Role · 권한 설계
강조	HTTPS · API 토큰 · RBAC · ABAC · ReBAC
실습	<code>run-chapter12-labs.sh</code> (8단계)

8장 403의 **정답**을 이번 장에서 설계·검증한다.

8장 → 12장 연결

8장	12장
Pod에서 <code>curl</code> + HTTPS + Bearer	같은 경로를 의도적으로 설계
<code>nodes</code> list → 403	Role + Binding 으로 허용 범위 정의
Downward API = 내 메타	RBAC = 클러스터 API 행위

한 줄: 8장은 “붙을 수 있다”, 12장은 “**뭘 허용할지**”다.

12.1 인증 (Authentication)

Users · ServiceAccount

인증 vs 인가 — 비교가 아니라 **순서**

RBAC·ABAC·ReBAC처럼 **고르는 모델**이 아니라, **모든 요청이 차례로 거치는 두 단계**다.

순서	영문	질문	실패 (대략)
1	인증 Authentication	누구인가?	401 Unauthorized
2	인가 Authorization	이 API 허용?	403 Forbidden

1번 통과 후에만 2번(RBAC 등) 평가 · 워크로드 주체 = **ServiceAccount**

HTTPS — 왜 API는 HTTPS인가

HTTPS가 하는 일	설명
암호화	요청·응답을 네트워크에서 읽기 어렵게
서버 인증	상대가 진짜 API 서버 인지 — Pod의 <code>ca.crt</code> 로 검증
기본 포트	443 (<code>https://kubernetes.default.svc</code>)

`--cacert .../ca.crt` **없이** TLS 검증을 끄면 **중간자 공격**에 취약 — 실습·운영 모두 CA 사용.

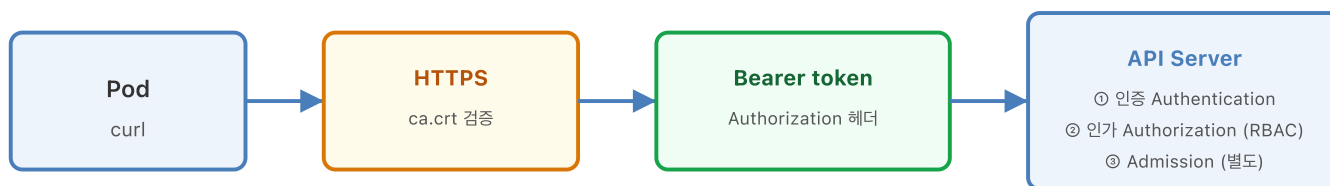
HTTPS — Pod 안에서의 주소

- `https://kubernetes.default.svc` (또는 `https://10.96.0.1:443`)
- 환경 변수: `KUBERNETES_SERVICE_HOST` / `KUBERNETES_SERVICE_PORT`
- `http://` 만 쓰면 리다이렉트·거부 등 **환경마다 실패** (실습 2절에서 예러 확인)

```
curl --cacert /var/run/secrets/kubernetes.io/serviceaccount/ca.crt \  
-H "Authorization: Bearer $(cat ../token)" \  
https://kubernetes.default.svc/api/v1/nodes
```


HTTPS + 토큰 + 인증·인가 흐름

Pod → API Server 요청 흐름



결과

401 Unauthorized

토큰·CA 문제

403 Forbidden

인증 OK · RBAC 거부

200 OK

Rule + Binding 허용

403 (8장·12장): TLS·토큰은 유효 → **RBAC** 이 `list nodes` 거부

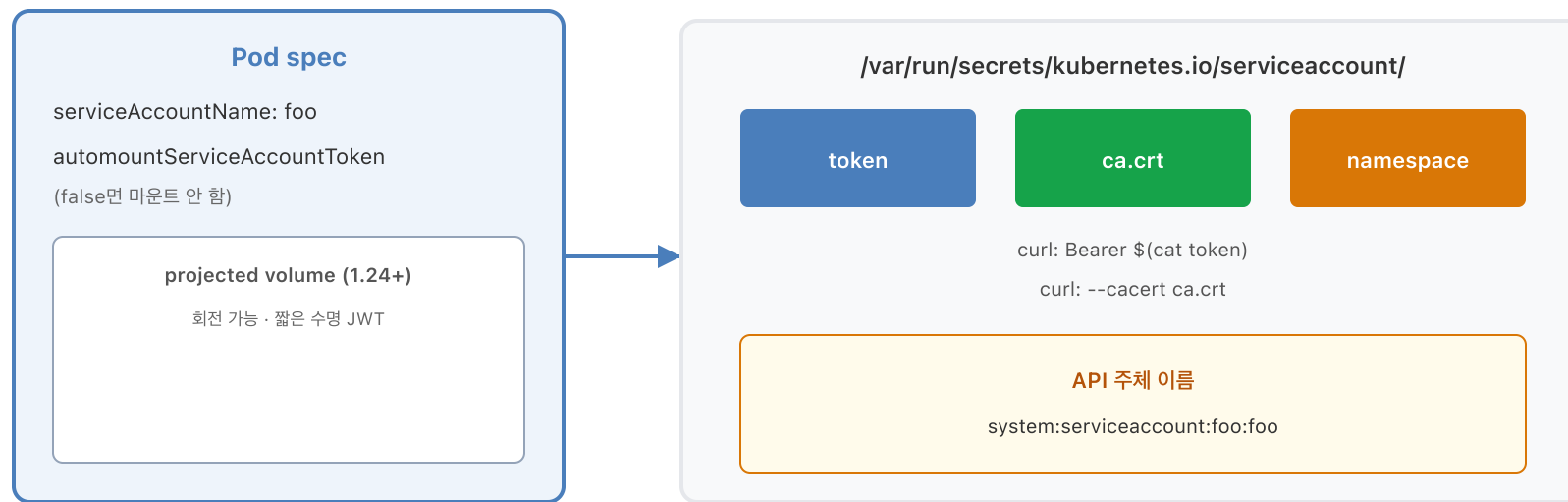
API 토큰 (ServiceAccount) — 개념

- Pod에 마운트된 `token` 파일 → `Authorization: Bearer <값>`
- API 서버가 주체를 식별: `system:serviceaccount:<ns>:<sa>`
- 형식: 보통 **JWT** (서명·만료·audience 등 메타데이터)

토큰이 유효 ≠ 권한 있음 — RBAC Rule이 없으면 **403**

API 토큰 — 마운트 구조

ServiceAccount 토큰 마운트



`spec.serviceAccountName` — **Pod 생성 시** 지정, 이후 변경 불가 (책)

API 토큰 — 세 파일의 역할

파일	용도
<code>token</code>	Bearer 인증 헤더
<code>ca.crt</code>	API 서버 TLS 신뢰 앵커
<code>namespace</code>	이 토큰의 네임스페이스 (URL 조합 시 참고)

경로: `/var/run/secrets/kubernetes.io/serviceaccount/`

API 토큰 — 1.24+ 클러스터 변화

시기	패턴
예전	SA 생성 시 장기 Secret 자동 생성·마운트
1.24+	장기 토큰 Secret 자동 생성 비활성 방향
권장	projected volume — 회전 가능·짧은 수명 JWT

개념은 동일: Pod는 **파일에서 토큰을 읽어** API에 제시한다.

ServiceAccount 생성 · Pod에 할당

```
# serviceaccount-foo.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: foo
  namespace: foo
```

```
# curl-custom-sa.yaml
spec:
  serviceAccountName: foo # default 대신 전용 SA
```

운영: 앱마다 전용 **SA** + 최소 **Role** — default SA에 cluster-admin 금지

SA의 imagePullSecrets (책 12.1)

- **7장**에서 Pod마다 `imagePullSecrets` 를 넣었음
- **12장**: SA 리소스에 `imagePullSecrets` 를 두면, 그 SA를 쓰는 Pod에 **자동 적용**
- 파일: `sa-image-pull-secrets.yaml` (프라이빗 레지스트리 인증 분리)

API 불필요 워크로드 — 토큰 마운트 차단

```
spec:  
  automountServiceAccountToken: false
```

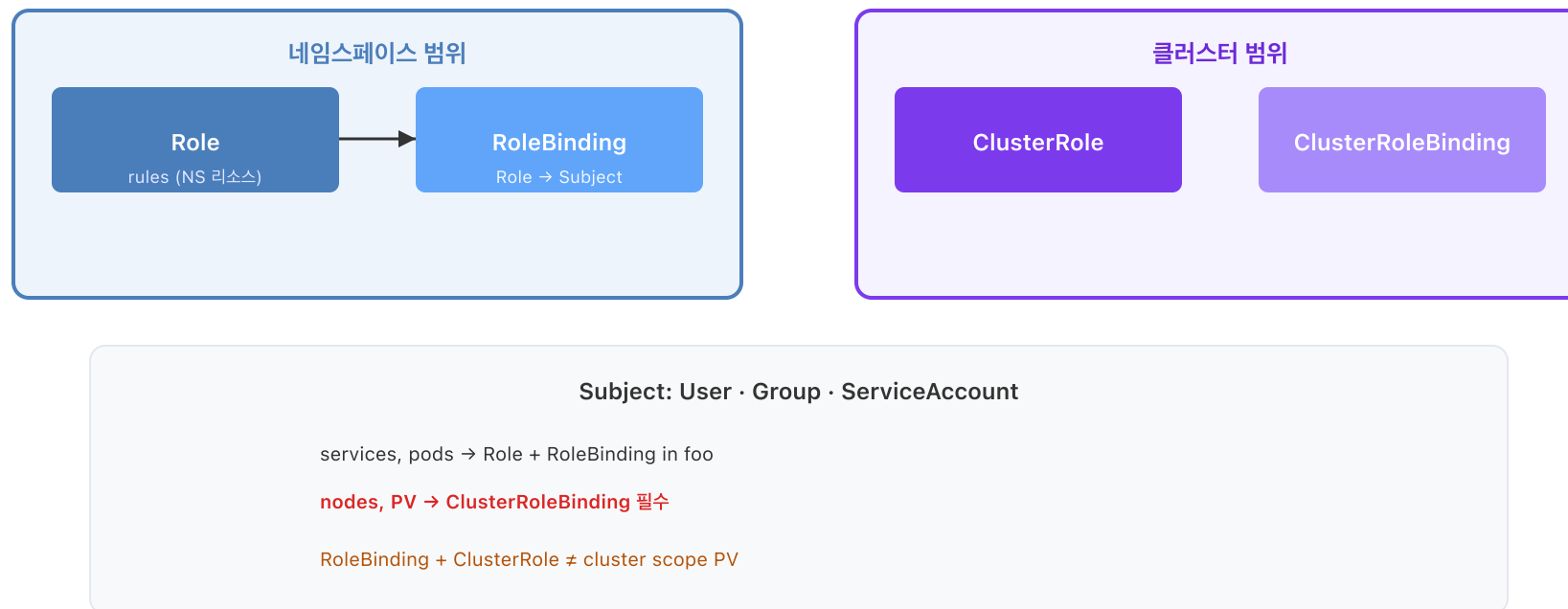
- API를 **전혀** 쓰지 않는 앱은 토큰 파일 자체를 **마운트하지 않음**
- 공격면(토큰 유출) 축소 — **최소 권한**의 한 축

12.2 인가 (Authorization) — RBAC

Role · RoleBinding · ClusterRole · ClusterRoleBinding

RBAC 4가지 리소스

RBAC 4가지 리소스



Subject: User · Group · ServiceAccount

RBAC Rule — YAML 구조

```
rules:
- apiGroups: [""]          # core API → 빈 문자열
  resources: ["services"]  # 복수형 (services, pods, nodes)
  verbs: ["get", "list"]   # get, list, watch, create, update, patch, delete
  # resourceName: ["demo-svc"] # 특정 이름만 허용
```

필드	의미
apiGroups	"" = core, apps = Deployment 등
resources	API 리소스 이름 (복수형)
verbs	허용 동작

verbs — 자주 쓰는 조합

verb	의미	실습 예
<code>get</code>	단일 리소스 조회	—
<code>list</code>	컬렉션 목록	<code>services, pods, nodes, PV</code>
<code>watch</code>	변경 스트림	<code>컨트롤러·operator</code>
<code>create</code> / <code>update</code> / <code>patch</code> / <code>delete</code>	쓰기	<code>can-i create</code> → no 확인

```
kubectl auth can-i list services --as=system:serviceaccount:foo:foo -n foo
kubectl auth can-i --list --as=system:serviceaccount:foo:foo -n foo
```

네임스페이스 vs 클러스터 스코프

리소스	Role + RoleBinding (NS)	ClusterRoleBinding
services, pods in foo	✓	(다른 패턴도 가능)
nodes	✗ cluster scope	✓ 필수
persistentvolumes	✗	✓ 필수

8장 403: nodes at **cluster scope** — NS Role만으로는 부족

RoleBinding이 ClusterRole을 참조할 때

- **RoleBinding** 의 `roleRef.kind: ClusterRole` 은 가능
- 효과: 해당 **RoleBinding**의 namespace 안 리소스에만 규칙 적용
- **cluster scope 리소스(PV, nodes)** 는 **ClusterRoleBinding** 필요

책 12.2.4 함정: ClusterRole + **RoleBinding**만 → PV list 403

PV Binding 합정 vs 해결 (12.2.4)

PV list — Binding 합정 (책 12.2.4)

잘못된 패턴

ClusterRole pv-reader
+ RoleBinding (namespace foo)

PV list → 403

cluster scope 리소스

올바른 패턴

ClusterRole pv-reader
+ ClusterRoleBinding

PV list → 200

SA foo에 cluster-wide grant

RoleBinding은 해당 NS 안 리소스에만 효과가 제한될 수 있음

1. `rolebinding-pv-wrong.yaml` → **403**
2. 삭제 후 `clusterrolebinding-pv-reader.yaml` → **200**

기본 ClusterRole (참고)

이름	대략
<code>view</code>	읽기 위주
<code>edit</code>	읽기+쓰기 (일부 제한)
<code>admin</code>	NS 관리에 가깝게
<code>cluster-admin</code>	클러스터 전권 — SA에 주면 사고

Minikube: `default/default` 가 cluster-admin 인 경우 **흔함** — 운영 환경과 다름

권한 부여 원칙 (책 12.2 · Grant wisely)

1. **필요한 API만** — `list nodes` 가 없으면 굳이 열지 않음
2. **전용 SA** — 앱·팀 단위 분리
3. **Binding 범위** — NS vs cluster 구분
4. **정기 점검** — `cluster-admin` 에 묶인 SA 목록
5. **can-i로 검증** — YAML 적용 전·후

```
kubectl create role service-reader --verb=get,list --resource=services -n foo  
kubectl create clusterrolebinding pv-test --clusterrole=pv-reader --serviceaccount=foo:foo
```

ABAC · ReBAC

RBAC을 넘는 정책 모델

RBAC — Kubernetes 기본 인가 (본체)

항목	내용
기준	역할(Role) + <code>apiGroups</code> / <code>resources</code> / <code>verbs</code>
부여	RoleBinding · ClusterRoleBinding → Subject
K8s	API 서버 authorization mode = RBAC (일반 클러스터)
장점	단순·표준· <code>kubectl auth can-i</code> 검증
한계	"오늘만", "이 라벨일 때만" → Rule만으로 부족

오늘 실습의 100% = RBAC

ABAC — Attribute-Based Access Control

속성으로 허용/거부:

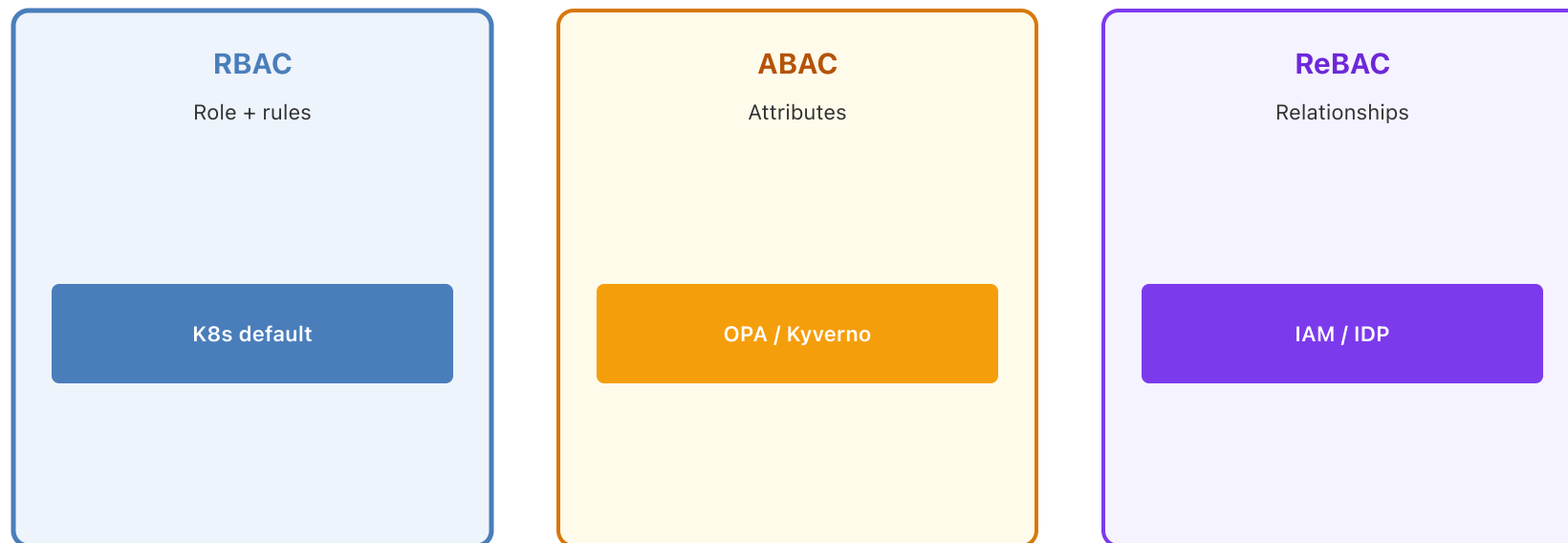
- 요청자 IP · 시간대 · 리소스 **라벨** · NS 라벨 `team=payments`
- “이 **이미지 태그**만”, “**privileged** 컨테이너 금지”

K8s 코어	실무 보완
ABAC 정책 파일 모드 비권장 ·레거시	Kyverno, OPA Gatekeeper
	클라우드 IAM condition

기억: K8s 안 큰 경계 = **RBAC**, 그 위 세밀 정책 = **ABAC**에 가까운 도구

RBAC · ABAC · ReBAC — 한눈에

RBAC vs ABAC vs ReBAC



	RBAC	ABAC	ReBAC
기준	역할·verb	속성	관계 그래프
K8s 코어	✓ 기본	✗	✗
도구 예	Role, Binding	Kyverno, OPA	IAM, SpiceDB , Zanzibar

ReBAC — Relationship-Based Access Control

관계로 권한 결정:

- “User A는 Project P의 **member** → Resource R **read**”
- 대표: Google **Zanzibar** 모델

Kubernetes	한계
RoleBinding = SA ↔ Role 1단계	팀·프로젝트 계층·상속 표현 부족
	조직 IAM·IDP(Okta)와 연동이 실무

ReBAC 은 플랫폼·IDP·전용 엔진에서 — **Pod SA RBAC**과 병행 이해

세 모델 — 실무에서 같이 쓰는 법

[사람·CI] → 클라우드 IAM / IDP (ReBAC·ABAC에 가까움)
|
[Pod SA] → Kubernetes RBAC (오늘 실습)
|
[클러스터] → Kyverno/OPA (ABAC형 정책) + NetworkPolicy 등

12장 시험·면접: “K8s 기본 인가 = RBAC”, “더 세밀 = ABAC 도구”, “조직 관계 = ReBAC·IAM”

실습 (`run-chapter12-labs.sh`)

8단계 — HTTPS · 토큰 · RBAC 전 구간

사전: `podman machine start` → `minikube start`

실습 1-2 · 토큰 경로 · HTTPS · 403

```
kubectrl apply -f namespace-foo.yaml -f serviceaccount-foo.yaml -f curl-custom-sa.yaml
ls /var/run/secrets/kubernetes.io/serviceaccount/ # token, ca.crt, namespace
curl --cacert .../ca.crt -H "Authorization: Bearer $(cat .../token)" \
  https://kubernetes.default.svc/api/v1/nodes
```

관찰	의미
세 파일 존재	SA 토큰 마운트 OK
403 + system:serviceaccount:foo:foo	인증 OK, 인가(RBAC) 거부
HTTP만 호출 시 에러	HTTPS + ca.crt 필요

실습 3-4 · can-i · services Role

```
kubectl auth can-i --list --as=system:serviceaccount:foo:foo -n foo # 거의 비어 있음
kubectl apply -f service-reader.yaml -f rolebinding-service-reader.yaml -f demo-service.yaml
kubectl auth can-i list services --as=system:serviceaccount:foo:foo -n foo # yes
kubectl auth can-i create services --as=system:serviceaccount:foo:foo -n foo # no
```

API: `.../namespaces/foo/services` → **items에 demo-svc** (list 권한 증명)

실습 5 · pods — 같은 NS Role

```
kubectl apply -f pod-reader.yaml -f rolebinding-pod-reader.yaml
kubectl auth can-i list pods --as=system:serviceaccount:foo:foo -n foo
# GET .../namespaces/foo/pods → 200
```

NS Role 으로 **같은 namespace** Pod만 list — cluster 전체 Pod 아님

실습 6 · nodes — ClusterRoleBinding

```
kubectl apply -f node-lister.yaml -f clusterrolebinding-node-lister.yaml
kubectl auth can-i list nodes --as=system:serviceaccount:foo:foo
# GET .../api/v1/nodes → 200 NodeList
```

8장과 동일 URL — 차이는 **ClusterRoleBinding** 추가뿐

실습 7 · PV 함정 → CRB (12.2.4)

```
kubectl apply -f pv-reader.yaml -f rolebinding-pv-wrong.yaml
# PV list → 403 (RoleBinding + ClusterRole)
kubectl delete rolebinding pv-test-wrong -n foo
kubectl apply -f clusterrolebinding-pv-reader.yaml
# PV list → 200
```

교훈: cluster scope = **ClusterRoleBinding**

실습 8 · 최소 권한 · cluster-admin 점검

```
kubectl auth can-i list nodes --as=system:serviceaccount:default:default # no
# cluster-admin 에 묶인 ServiceAccount 목록 스크립트 출력
```

비교	결과
foo:foo	nodes yes (우리가 부여)
default:default in foo	nodes no (다른 SA)
cluster-admin SA	운영 사고 후보

정리(선택): `kubectl delete namespace foo`

핵심 한 문장

HTTPS + ca.crt 로 서버를 검증하고, **Bearer 토큰** 으로 **누구인지** 증명한 뒤, **RBAC** 으로 **필요한 API만** 연다.

ABAC / ReBAC 은 RBAC만으로 부족할 때 **도구·플랫폼**으로 보완한다.

자료: `CHAPTER12-Study.md` · `run-chapter12-labs.sh`